

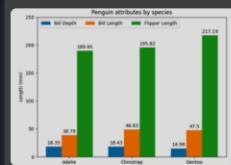
Python Course for Physicists (And everyone else!) Chunk 9: Matplotlib

By Jonathan Bollig

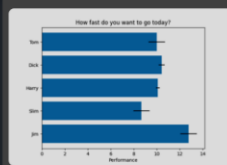
09.10.2025

Visualizing your Data: Matplotlib

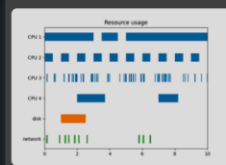
- Most used plotting and visualization library
- Examples



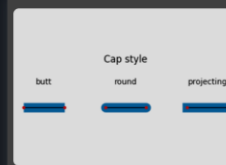
Grouped bar chart
with labels



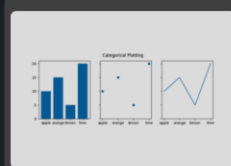
Horizontal bar chart



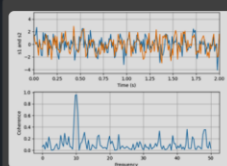
Broken horizontal
bars



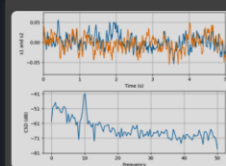
CapStyle



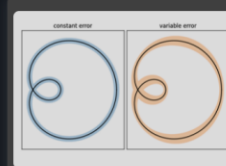
Plotting categorical
variables



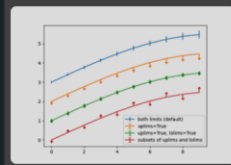
Plotting the
coherence of two
signals



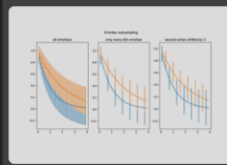
Cross spectral
density (CSD)



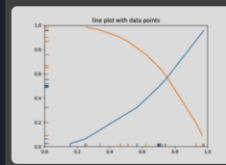
Curve with error
band



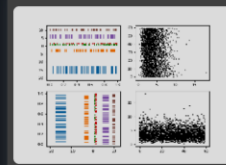
Errorbar limit
selection



Errorbar
subsampling



EventCollection
Demo



Eventplot demo

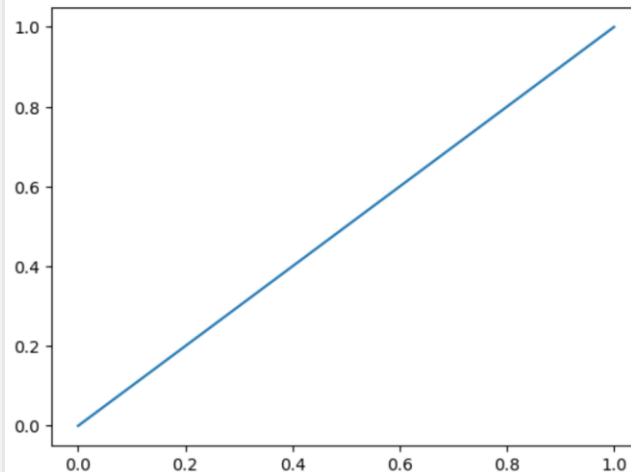
Live Coding: Getting Started

- Anaconda Powershell Prompt: conda install matplotlib (See chunk 7 – imports for details)
- Restart Kernel (if used before)
- `import matplotlib.pyplot as plt`
- Try:

submodule

submodule

```
import matplotlib.pyplot as plt  
plt.plot([0, 1])  
plt.show()
```



Two Types of Plotting

- Two ways to use matplotlib: Quick & dirty or the clean way.
- Quick & dirty:
 - Less code
 - Faster to learn
 - Limited
- Clean way:
 - A little more boilerplate code
 - Slightly different coding “style”
 - Better for complex plotting (e. g. multiple plots next to each other)
- Problem: Codes are similar, but slightly different
→ Be aware when googling stuff

```
import matplotlib.pyplot as plt

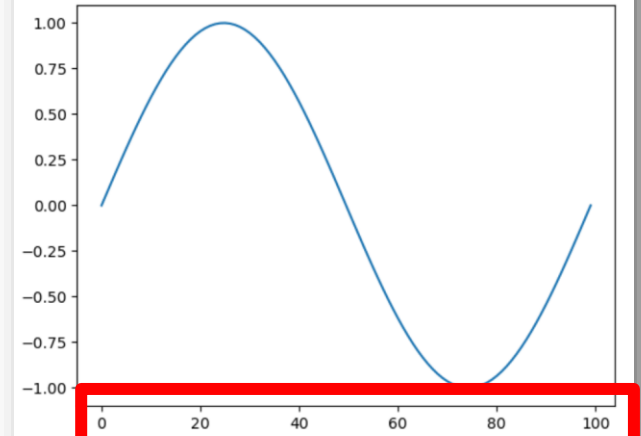
plt.plot([0, 1])
plt.xlabel("x")

fig, ax = plt.subplots()
ax.plot([0, 1])
ax.set_xlabel("x")
```

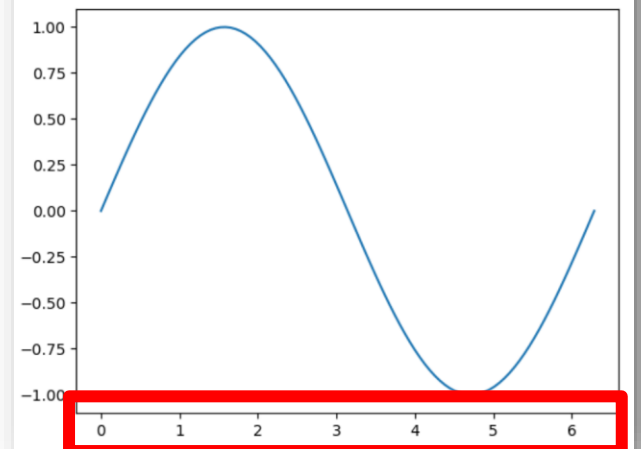
Live Coding: Basics of Plotting

- `plt.plot()`: Can be used in two ways
 - Passing y-data → x-Axis is the index
 - Passing x- and y-data: → normal axis
- `plt.show()`: Generates the plot
 - Mostly works without `plt.show()` but good practice to use it.

```
x = np.linspace(0, 2 * np.pi, 100)  
y = np.sin(x)  
plt.plot(y)  
plt.show()
```



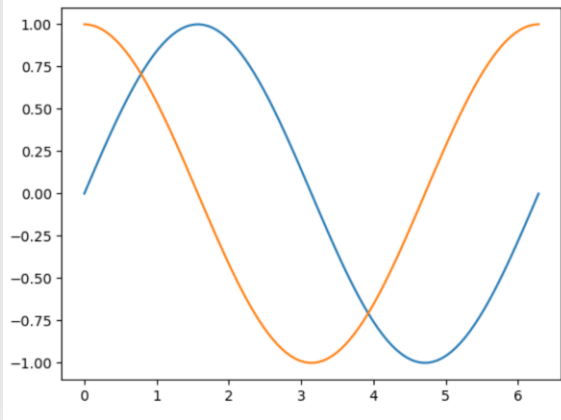
```
x = np.linspace(0, 2 * np.pi, 100)  
y = np.sin(x)  
plt.plot(x, y)  
plt.show()
```



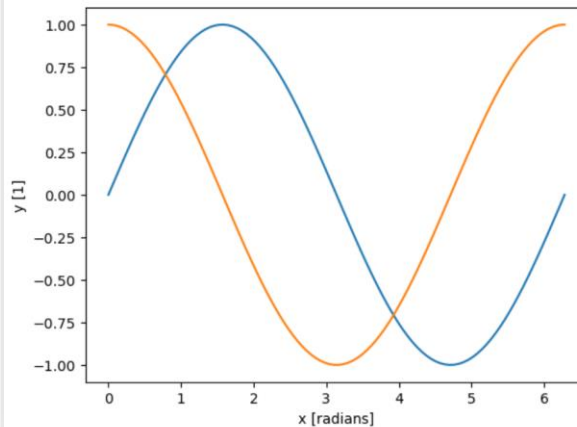
Live Coding: Adding to your Plot

- More data: repeat `plt.plot()`
- Add x- and y-label to your plot: `plt.xlabel("x")` and `plt.ylabel("y")`
- Add a background grid: `plt.grid()`

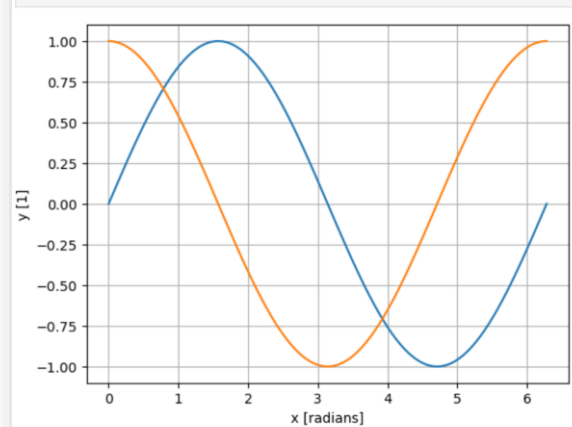
```
x = np.linspace(0, 2 * np.pi, 100)
y1 = np.sin(x)
y2 = np.cos(x)
plt.plot(x, y1)
plt.plot(x, y2)
plt.show()
```



```
x = np.linspace(0, 2 * np.pi, 100)
y1 = np.sin(x)
y2 = np.cos(x)
plt.plot(x, y1)
plt.plot(x, y2)
plt.xlabel("x [radians]")
plt.ylabel("y [1]")
plt.show()
```



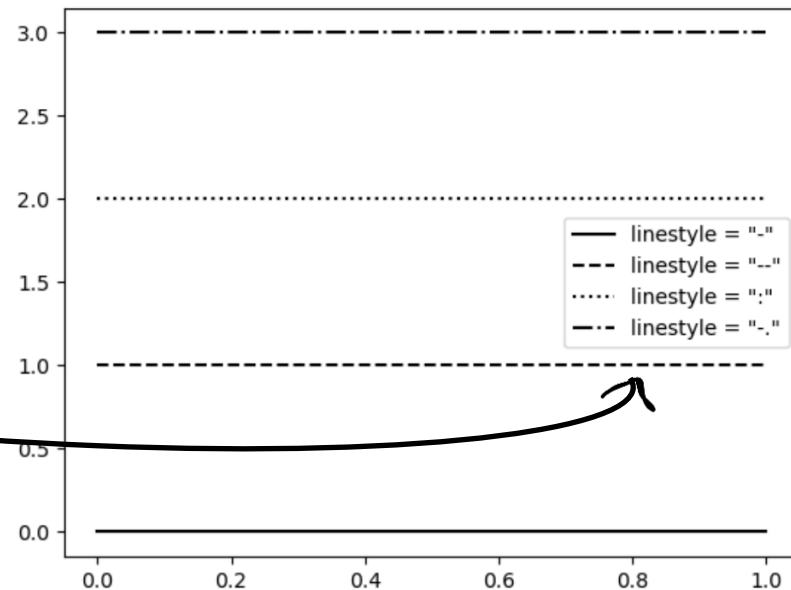
```
x = np.linspace(0, 2 * np.pi, 100)
y1 = np.sin(x)
y2 = np.cos(x)
plt.plot(x, y1)
plt.plot(x, y2)
plt.xlabel("x [radians]")
plt.ylabel("y [1]")
plt.grid()
plt.show()
```



Options for plt.plot()

- Besides x- and y-data, we can pass more optional parameters to the plot() function. To specify what we are passing, we can use “=”.
- Example: plt.plot(x, y, c=“lightgrey”)
- Optional arguments:
 - c → Color of the graph
 - linestyle → Style of the line
 - label → Strings get imaged

→ Need to call legend()



Good Luck with your Tasks 😊

